

Python E-Course

Module 4 — Functions

Detailed Notes

Module Overview

Level: Beginner

Duration: ~1.5 weeks (10-12 hours)

Prerequisites: Modules 1-3

What You'll Learn

- Define and call functions with `def`.
- Use parameters, default values, keyword arguments, and `args/*kwargs`.
- Return single and multiple values.
- Understand local vs global scope.
- Use `lambda` for tiny anonymous functions.
- Document functions with docstrings.

Lessons

#	Lesson	Duration
1	Defining and Calling Functions	30 min
2	Parameters and Return Values	35 min
3	Default & Keyword Arguments	30 min
4	<code>args</code> and <code>*kwargs</code>	30 min
5	Scope and Lifetime	30 min
6	Lambdas and Docstrings	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 5: Data Structures — lists, tuples, sets, dictionaries.

Lesson 1: Defining and Calling Functions

Module: 4 — Functions

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Define a function with `def`.
- Call a function and use its return value.
- Distinguish parameters from arguments.

Prerequisites

- Module 3 — All lessons

1. Why This Matters

A function packages code so you can name it, run it from anywhere, and avoid repeating yourself. Every non-trivial program is a network of functions calling each other. Until now your scripts have been one long sequence — functions are how you start writing programs that scale.

2. Concept

2.1 The `def` keyword

```
def greet():  
    print("Hello!")
```

- `def` introduces a function definition.
- `greet` is the function's name.
- `()` is the parameter list (empty here).
- `:` starts the body, which must be indented.

2.2 Calling a function

```
greet()
```

The function name + `()` runs the body. Without `()`, you just refer to the function object — you don't run it.

2.3 Parameters vs arguments

- Parameter — a name in the function definition.
- Argument — a value you pass in when calling.

```
def greet(name):          # name is a parameter  
    print(f"Hi, {name}")  
  
greet("Maya")             # "Maya" is an argument
```

2.4 Return values

A function can send a value back with return:

```
def double(x):  
    return x * 2  
  
result = double(5)    # result is 10
```

A function without return (or with bare return) returns None.

2.5 Why functions

- Reuse: write logic once, call from many places.
- Naming: a good function name documents intent.
- Testing: small functions are easier to verify than long scripts.
- Abstraction: callers don't need to know the implementation.

3. Code Examples

Example 1: No parameters, no return

```
def banner():  
    print("=" * 20)  
    print("  MY PROGRAM")  
    print("=" * 20)  
  
banner()  
banner()
```

Expected Output:

```
=====
  MY PROGRAM
=====
=====
  MY PROGRAM
=====
```

Explanation: Defined once; called twice.

Example 2: With parameter

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Sam")  
greet("Maya")
```

Expected Output:

```
Hello, Sam!  
Hello, Maya!
```

Explanation: Each call passes a different argument.

Example 3: With return value

```
def square(n):  
    return n * n  
  
print(square(4))  
print(square(10))  
total = square(3) + square(5)  
print(total)
```

Expected Output:

```
16  
100  
34
```

Explanation: return sends the value back; the caller uses it like any other value.

Example 4: No return → None

```
def shout(msg):  
    print(msg.upper())  
  
result = shout("hello")  
print(result)
```

Expected Output:

```
HELLO  
None
```

Explanation: shout prints but doesn't return. The default return is None.

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting () when calling	Nothing runs; you just get the function object	Add ()
Confusing print with return	Caller can't use the value	Use return to send back
Calling before defining	NameError	Define functions above their callers (or at top of file)
Defining inside a loop accidentally	Redefined each iteration	Move def outside the loop

5. Practice Tasks

- Write hello() that prints Hi!. Call it three times.
- Write cube(n) that returns n ** 3. Print cube(4).
- Write welcome(name) that prints Welcome, NAME!.

6. Key Takeaways

- def name(params): defines; name(args) calls.
- Parameters are placeholders; arguments are values you pass.

- return sends a value back; no return → None.
- Functions let you name, reuse, and test pieces of logic.

7. What's Next

Next: deeper on parameters and return values — multiple parameters, multiple returns, and best practices.

Lesson 2: Parameters and Return Values

Module: 4 — Functions

Duration: 35 minutes

Difficulty: Beginner

Learning Objectives

- Define functions with multiple parameters.
- Return multiple values via a tuple.
- Use early return to handle special cases.

Prerequisites

- Module 4 Lesson 1

1. Why This Matters

Real functions take multiple inputs and often produce multiple outputs (e.g., "divide and return quotient + remainder"). Knowing how to design parameter lists and return values is the single biggest skill for writing clean code.

2. Concept

2.1 Multiple parameters

```
def add(a, b):  
    return a + b  
  
print(add(3, 5))    # 8
```

Pass arguments in the same order as the parameters (positional).

2.2 Returning multiple values

Return a tuple (covered formally in Module 5). Python lets you list values separated by commas:

```
def divmod_(a, b):  
    return a // b, a % b  
  
q, r = divmod_(17, 5)  
print(q, r)    # 3 2
```

The values are "unpacked" into separate variables on the caller side.

2.3 Early return

Return early to handle special cases — keeps the main path uncluttered:

```
def safe_divide(a, b):  
    if b == 0:  
        return None  
    return a / b
```

2.4 One responsibility per function

A function should do one thing. Resist the urge to do "load data, transform, format, and print" in one giant function. Split it.

2.5 Returning vs printing

These are different:

```
def print_double(x):
    print(x * 2)          # prints, doesn't return

def double(x):
    return x * 2          # returns, doesn't print
```

`double(3) + 1` works; `print_double(3) + 1` raises `TypeError` because `print_double` returns `None`.

3. Code Examples

Example 1: Two parameters

```
def rectangle_area(width, height):
    return width * height

print(rectangle_area(4, 5))
```

Expected Output: 20

Example 2: Multiple returns

```
def min_max(a, b, c):
    return min(a, b, c), max(a, b, c)

lo, hi = min_max(3, 9, 1)
print(lo, hi)
```

Expected Output: 1 9

Explanation: Function returns two values; caller unpacks them.

Example 3: Early return for edge case

```
def safe_divide(a, b):
    if b == 0:
        return None
    return a / b

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Expected Output:

```
5.0
None
```

Explanation: Special case handled at the top; main path stays one line.

Example 4: Combining outputs

```
def stats(a, b):  
    total = a + b  
    diff = a - b  
    avg = (a + b) / 2  
    return total, diff, avg  
  
s, d, m = stats(10, 4)  
print(f"sum={s}, diff={d}, mean={m}")
```

Expected Output: sum=14, diff=6, mean=7.0

Explanation: Three computed values returned together.

4. Common Mistakes

Mistake	What Happens	Fix
Mixing argument order	Wrong values used	Use keyword args (Lesson 3) for clarity
Returning when you meant print	Nothing displayed	Print at the call site, or call from a printing context
Doing too much in one function	Hard to test/maintain	Split by responsibility
Forgetting to unpack	print(stats(10, 4)) shows a tuple	Use s, d, m = stats(...)

5. Practice Tasks

- Write multiply(a, b) returning a * b. Test with multiply(7, 6).
- Write min_max(a, b, c) returning min and max as a tuple. Test with values of your choice.
- Write safe_sqrt(x) that returns None if x < 0, else x ** 0.5.

6. Key Takeaways

- Parameters listed in def, arguments passed in order at call.
- Return multiple values as a tuple; unpack with a, b = func().
- Early returns for edge cases keep the main flow simple.
- One responsibility per function.

7. What's Next

Default values and keyword arguments — make function calls more flexible and self-documenting.

Lesson 3: Default & Keyword Arguments

Module: 4 — Functions

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Give parameters default values.
- Call functions using keyword arguments for clarity.
- Avoid the mutable default argument trap.

Prerequisites

- Module 4 Lesson 2

1. Why This Matters

Default arguments let you write one flexible function instead of multiple variants. Keyword arguments make calls self-documenting — `connect(host="db.local", port=5432)` reads better than `connect("db.local", 5432)`. Together, they make APIs much friendlier.

2. Concept

2.1 Default values

Give a parameter a default with `=`:

```
def greet(name, greeting="Hello"):
    print(f'{greeting}, {name}!')

greet("Sam")           # uses default
greet("Sam", "Hi")     # overrides
```

Parameters with defaults must come after parameters without defaults:

```
# BAD
def f(x=1, y):          # SyntaxError
    ...

# GOOD
def f(y, x=1):
    ...
```

2.2 Keyword arguments at the call site

You can pass arguments by name in any order:

```
greet(name="Sam", greeting="Hi")
greet(greeting="Hi", name="Sam")  # same effect
```

Mixing positional and keyword: positional first, keyword after.

```
greet("Sam", greeting="Hi") # OK
greet(name="Sam", "Hi")     # SyntaxError
```

2.3 When to use which

- Few parameters, obvious order → positional: `add(3, 5)`.
- Many parameters, or boolean flags → keyword: `connect(host=..., port=..., ssl=True)`.

2.4 Mutable default argument trap

Never use a mutable object (list, dict, set) as a default:

```
# WRONG – surprising bug
def append_to(item, target=[]):
    target.append(item)
    return target

print(append_to(1)) # [1]
print(append_to(2)) # [1, 2] – SAME list reused!
```

Fix: use `None`, then build the list inside.

```
def append_to(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

2.5 Self-documenting calls

Keyword args label what each value means:

```
make_account(name="Maya", balance=0, currency="INR", overdraft=False)
```

Reads better than `make_account("Maya", 0, "INR", False)`.

3. Code Examples

Example 1: Default greeting

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Sam")
greet("Maya", "Hi")
greet("Lee", greeting="Hola")
```

Expected Output:

```
Hello, Sam!
Hi, Maya!
Hola, Lee!
```

Explanation: Default used when omitted; overridden by position or keyword.

Example 2: Calculator with default operation

```
def calc(a, b, op="add"):
    if op == "add":
        return a + b
    if op == "sub":
        return a - b
    if op == "mul":
        return a * b
    return None

print(calc(3, 5))
print(calc(3, 5, op="mul"))
```

Expected Output:

```
8
15
```

Example 3: Mutable default trap

```
def add_item(item, bag=None):
    if bag is None:
        bag = []
    bag.append(item)
    return bag

print(add_item("apple"))
print(add_item("bread"))
```

Expected Output:

```
['apple']
['bread']
```

Explanation: Fresh list each call because we used None as the default.

Example 4: Keyword call for clarity

```
def book_flight(origin, destination, date, seats=1, refundable=False):
    print(f"{origin} -> {destination} on {date}, {seats} seat(s),\nrefundable={refundable}")

book_flight(origin="DEL", destination="BLR", date="2026-06-01", seats=2)
```

Expected Output:

```
DEL -> BLR on 2026-06-01, 2 seat(s), refundable=False
```

4. Common Mistakes

Mistake	What Happens	Fix
Defaults before non-defaults	SyntaxError	Put defaults last
Mutable default =[]	Shared across calls	Use =None; initialize inside
Positional after keyword	SyntaxError	Keyword args go last

Wrong keyword name	TypeError: unexpected keyword argument	Check the parameter name
--------------------	--	--------------------------

5. Practice Tasks

- Write `power(base, exp=2)` returning `base ** exp`. Test with one and two args.
- Fix this buggy function:

```
python def collect(x, into=[]): into.append(x) return into
```

- Call `book_flight(...)` from Example 4 with `refundable=True` and 3 seats.

6. Key Takeaways

- Defaults: `def f(x, y=10):` — caller can omit `y`.
- Keyword args at the call site improve readability.
- Never use mutable objects as defaults — use `None`.

7. What's Next

`args` and `*kwargs` — accept variable numbers of positional and keyword arguments.

Lesson 4: args and *kwargs

Module: 4 — Functions

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Accept any number of positional arguments with *args.
- Accept any number of keyword arguments with **kwargs.
- Unpack iterables and dicts when calling functions.

Prerequisites

- Module 4 Lesson 3

1. Why This Matters

Sometimes you don't know in advance how many arguments a function should accept. `print()` itself takes any number — that's args. Frameworks pass settings as keyword bags — that's *kwargs. Knowing both lets you read library code and write flexible functions.

2. Concept

2.1 *args

Collects extra positional arguments into a tuple:

```
def total(*nums):
    return sum(nums)

print(total(1, 2, 3))      # 6
print(total(10, 20))      # 30
print(total())             # 0
```

The name args is convention — nums is fine. The is what matters.

2.2 **kwargs

Collects extra keyword arguments into a dict:

```
def show(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

show(name="Sam", age=30, city="Mumbai")
```

Output:

```
name: Sam
age: 30
city: Mumbai
```

(Dicts and .items() are formally covered in Module 5.)

2.3 Combining everything

```
def f(a, b, *args, x=10, **kwargs):  
    ...
```

Order: regular → args → keyword-only → *kwargs. You'll rarely need all four.

2.4 Unpacking at the call site

The mirror operation — spread a list/tuple as positional args or a dict as keyword args:

```
def greet(first, last):  
    print(f"Hi, {first} {last}")  
  
names = ("Sam", "Lee")  
greet(*names)           # same as greet("Sam", "Lee")  
  
info = {"first": "Sam", "last": "Lee"}  
greet(**info)           # same as greet(first="Sam", last="Lee")
```

3. Code Examples

Example 1: Sum any count

```
def total(*nums):  
    return sum(nums)  
  
print(total(1, 2, 3, 4, 5))  
print(total(10))  
print(total())
```

Expected Output:

```
15  
10  
0
```

Example 2: Keyword bag

```
def make_user(**fields):  
    for key, value in fields.items():  
        print(f" {key} = {value}")  
  
make_user(name="Maya", age=22, city="Delhi")
```

Expected Output:

```
name = Maya  
age = 22  
city = Delhi
```

Example 3: Unpacking a list

```
def add3(a, b, c):  
    return a + b + c
```

```
nums = [1, 2, 3]
print(add3(*nums))
```

Expected Output: 6

Explanation: `*nums` spreads the list as three positional args.

Example 4: Unpacking a dict

```
def book(title, author, year):
    print(f"{title} by {author} ({year})")

data = {"title": "1984", "author": "Orwell", "year": 1949}
book(**data)
```

Expected Output: 1984 by Orwell (1949)

Explanation: `**data` spreads dict entries as keyword arguments.

Example 5: Mix

```
def log(level, *messages, **meta):
    tag = meta.get("tag", "GENERAL")
    print(f"[{level}|{tag}]", *messages)

log("INFO", "user", "logged", "in", tag="AUTH")
```

Expected Output: [INFO|AUTH] user logged in

Explanation: `level` is positional, `messages` is the tuple of remaining positionals, `meta` is the kwargs dict. (`meta.get` covered in M5.)

4. Common Mistakes

Mistake	What Happens	Fix
Calling <code>total(1, 2, 3)</code> when defined <code>def total(a, b, c)</code>	Works but inflexible	Use <code>*args</code> for variable counts
Forgetting <code>*</code> when unpacking	Passes list as single arg	Use <code>*list_name</code>
Mixing wrong parameter order	<code>SyntaxError</code>	regular → args → kw-only → *kwargs

5. Practice Tasks

- Write `concat(*parts)` that joins all string args with a space and returns the result.
- Write `summary(**fields)` that prints each key: value on its own line.
- Given `args = (5, 10)`, call a function `add(a, b)` using `*args`.

6. Key Takeaways

- `*args` → variable positional → tuple inside the function.
- `**kwargs` → variable keyword → dict inside the function.
- `list` / `*dict` at the call site unpack into arguments.

7. What's Next

Scope and lifetime — where variables live and which functions can see them.

Lesson 5: Scope and Lifetime

Module: 4 — Functions

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Distinguish local, enclosing, global, and built-in scope (LEGB).
- Read and write global variables with global (and know why to avoid it).
- Understand a variable's lifetime — when it exists and when it disappears.

Prerequisites

- Module 4 Lesson 4

1. Why This Matters

Bugs from "I changed it inside the function but it didn't update outside" are constant for beginners. Knowing the rules for where variables live — and resisting global — is what separates clean code from spaghetti.

2. Concept

2.1 LEGB rule

Python looks up names in this order:

- Local — inside the current function.
- Enclosing — in the local scope of any enclosing function.
- Global — at the top level of the module.
- Built-in — names like print, len, range.

First match wins.

2.2 Local scope

Variables created inside a function exist only there:

```
def f():  
    x = 5  
    print(x)  
  
f()  
print(x)  # NameError: x is not defined
```

2.3 Reading globals

A function can read a top-level (global) variable:

```
PI = 3.14159
```

```
def area(r):  
    return PI * r * r
```

2.4 Writing globals — global

To reassign a top-level variable from inside, declare global:

```
counter = 0  
  
def increment():  
    global counter  
    counter += 1  
  
increment()  
print(counter)    # 1
```

Without global, counter += 1 would create a new local variable and fail at the += step.

Use global sparingly. Returning a value is almost always cleaner:

```
def increment(c):  
    return c + 1  
  
counter = increment(counter)
```

2.5 Lifetime

A local variable exists only while its function is running. When the function returns, the local namespace is destroyed.

2.6 Shadowing built-ins

Don't name your variable list, str, dict, type, sum. You shadow the built-in for the rest of the scope.

3. Code Examples

Example 1: Local stays local

```
def f():  
    x = 5  
    print("inside:", x)  
  
f()  
print("outside:", x)
```

Expected Output:

```
inside: 5  
NameError: name 'x' is not defined
```

Explanation: x only exists during f()'s execution.

Example 2: Reading global

```
TAX = 0.18

def with_tax(price):
    return price * (1 + TAX)

print(with_tax(100))
```

Expected Output: 118.0

Explanation: Functions can read top-level names without global.

Example 3: Writing global with global

```
counter = 0

def bump():
    global counter
    counter += 1

bump()
bump()
bump()
print(counter)
```

Expected Output: 3

Example 4: Refactor away from global

```
def bump(c):
    return c + 1

counter = 0
counter = bump(counter)
counter = bump(counter)
counter = bump(counter)
print(counter)
```

Expected Output: 3

Explanation: Same result, no global state. Easier to test.

Example 5: Built-in shadowing trap

```
sum = 100          # BAD – shadows built-in
nums = [1, 2, 3]
# print(sum(nums)) # TypeError: 'int' object is not callable
del sum           # restore built-in
print(sum(nums))  # 6
```

Explanation: Don't reuse names of built-ins as variables.

4. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

Assuming inner change affects outside	Variable stays unchanged	Return value and reassign at call site
Forgetting global when reassigning	UnboundLocalError	Add global, or better, return
Shadowing built-ins	Confusing later errors	Rename your variable
Using globals to share state across functions	Hard to test, fragile	Pass values as arguments

5. Practice Tasks

- Define a function that creates a local secret = 42 and try to print secret after calling it.
- Write make_counter() that uses a global count and returns the new value each call.
- Refactor the above to take and return count instead of using global.

6. Key Takeaways

- Name lookup follows LEGB: Local, Enclosing, Global, Built-in.
- Locals die when the function returns.
- Read globals freely; use global to reassign — but prefer returning.
- Don't shadow built-ins.

7. What's Next

The last lesson: lambdas (anonymous one-line functions) and docstrings (function documentation).

Lesson 6: Lambdas and Docstrings

Module: 4 — Functions

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Write lambda expressions for tiny anonymous functions.
- Know when not to use a lambda.
- Add docstrings to your functions and access them with `help()`.

Prerequisites

- Module 4 Lesson 5

1. Why This Matters

lambda lets you slot a tiny function inline — useful with `sorted`, `map`, `filter` (Module 11).

Docstrings turn your functions into self-documenting code; `help(func)` and IDE tooltips read them straight from there.

2. Concept

2.1 Lambda syntax

```
add = lambda a, b: a + b
print(add(3, 5))      # 8
```

- lambda keyword.
- Parameters before `:`.
- Single expression after `:` — its value is returned.

No `def`, no `return`, no name (unless you bind it).

2.2 When to use lambdas

Mainly as a one-off argument:

```
words = ["banana", "apple", "cherry"]
print(sorted(words, key=lambda w: len(w)))
# ['apple', 'banana', 'cherry'] — sorted by length
```

2.3 When NOT to use lambdas

If you'd give it a name longer than 5 letters or need more than one expression, use `def`:

```
# Bad
calc = lambda x: x ** 2 + 2 * x + 1

# Better
```

```
def calc(x):  
    return x ** 2 + 2 * x + 1
```

PEP 8 explicitly discourages assigning a lambda to a name — def is clearer.

2.4 Docstrings

A triple-quoted string as the first statement in a function:

```
def area(width, height):  
    """Return the area of a rectangle.  
  
    Both arguments must be non-negative numbers.  
    """  
    return width * height
```

Access with:

- `help(area)` — opens a help screen.
- `area.__doc__` — returns the string.

Convention: one short summary line, then optional details.

2.5 Docstring styles

Several conventions; pick one per project:

One-liner:

```
def double(x):  
    """Return twice x."""  
    return 2 * x
```

Google style:

```
def add(a, b):  
    """Add two numbers.  
  
    Args:  
        a: First number.  
        b: Second number.  
  
    Returns:  
        The sum.  
    """  
    return a + b
```

3. Code Examples

Example 1: Lambda with sorted

```
words = ["pear", "fig", "banana", "kiwi"]  
print(sorted(words, key=lambda w: len(w)))
```

Expected Output: ['fig', 'kiwi', 'pear', 'banana']

Explanation: Sort by length using a tiny key function.

Example 2: Lambda for inline math

```
square = lambda x: x * x
print(square(7))
```

Expected Output: 49

Explanation: Works, but `def square(x): return x * x` is clearer for a named function.

Example 3: Docstring

```
def greet(name):
    """Return a friendly greeting for the given name."""
    return f"Hello, {name}!"

print(greet("Maya"))
print(greet.__doc__)
help(greet)
```

Expected Output (help output truncated):

```
Hello, Maya!
Return a friendly greeting for the given name.
Help on function greet in module __main__:

greet(name)
    Return a friendly greeting for the given name.
```

Example 4: Multi-line docstring

```
def divide(a, b):
    """Return a / b.

    Returns None if b is zero.
    """
    if b == 0:
        return None
    return a / b

print(divide.__doc__)
```

Expected Output:

```
Return a / b.

Returns None if b is zero.
```

4. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

Using lambda for complex logic	Hard to read	Use def
Multiple statements in a lambda	SyntaxError	Refactor to def
Docstring not the first statement	<code>__doc__</code> is None	Place triple-quoted string immediately after def
Docstrings as # comments	<code>__doc__</code> is None; tools can't read	Use <code>"""..."""</code>

5. Practice Tasks

- Write lambda `x: x ** 3` and bind to `cube`. Test `cube(4)`.
- Sort `["zebra", "ant", "monkey"]` by length using a lambda.
- Add a one-line docstring to a function you wrote earlier this module and print `func.__doc__`.

6. Key Takeaways

- lambda args: expr makes a tiny anonymous function.
- Use lambdas inline as arguments; otherwise prefer def.
- Docstrings (`"""..."""`) as the first statement of a function are accessed by `help()` and `__doc__`.

7. What's Next

Module 4 ends here. Next: Module 5 — Data Structures: lists, tuples, sets, dictionaries — the workhorses of Python data.

Module 4 — Exercises

18 exercises (3 per lesson).

Lesson 4.1 — Defining

Exercise 1 (Easy)

Task: Define `hello()` printing Hi!. Call it 3 times.

Exercise 2 (Medium)

Task: Define `square(n)` returning $n * n$. Print squares of 1-5.

Exercise 3 (Hard)

Task: Define `is_even(n)` returning True/False. Use it to print even or odd for $n=1..10$.

Lesson 4.2 — Parameters/Return

Exercise 1 (Easy)

Task: Write `add(a, b)` returning sum. Print `add(7, 8)`.

Exercise 2 (Medium)

Task: Write `quad(a, b, c)` returning the discriminant $b^2 - 4a*c$. Test with $a=1$, $b=5$, $c=6$.

Exercise 3 (Hard)

Task: Write `min_max(nums)` returning a tuple of min and max. (You'll use args from L4 — fine; or take 3 fixed numbers.)

Lesson 4.3 — Defaults/Keyword

Exercise 1 (Easy)

Task: `power(base, exp=2)`. Test with `power(5)` and `power(5, 3)`.

Exercise 2 (Medium)

Task: `format_name(first, last, title="Mr.")`. Return "Mr. First Last".

Exercise 3 (Hard)

Task: Fix this bug:

```
def add_log(msg, log=[]):  
    log.append(msg)  
    return log
```

Lesson 4.4 — args / *kwargs

Exercise 1 (Easy)

Task: `total(*nums)` returns the sum. Test with 0, 1, and 5 arguments.

Exercise 2 (Medium)

Task: `make_url(**parts)` builds `"key=value&key=value..."` from `kwargs`.

Exercise 3 (Hard)

Task: Given `args = [3, 4]`, call `pow(*args)`. Predict output and verify.

Lesson 4.5 — Scope

Exercise 1 (Easy)

Task: Write a function that defines a local `x = 99` and prints it. Confirm `x` doesn't exist outside.

Exercise 2 (Medium)

Task: Use `global` to increment a top-level score from inside a function called 5 times. Print final score.

Exercise 3 (Hard)

Task: Refactor the above to pass score in and return it, avoiding `global`. Compare readability.

Lesson 4.6 — Lambdas/Docstrings

Exercise 1 (Easy)

Task: Bind `cube = lambda x: x ** 3`. Print `cube(4)`.

Exercise 2 (Medium)

Task: Sort `["cat", "elephant", "ox", "horse"]` by length using `sorted(..., key=lambda ...)`.

Exercise 3 (Hard)

Task: Add a docstring to your `min_max` function (from L2 Exercise 3). Print `min_max.__doc__`.

Module 4 — Quiz

10 multiple-choice questions covering Lessons 1-6.

Question 1

What does this print?

```
def f(x):  
    return x + 1  
print(f(5))
```

A. 5 B. 6 C. None D. SyntaxError

Answer: B — f(5) returns 6. (L1)

Question 2

What does a function without return return?

A. 0 B. "" C. None D. NaN

Answer: C (L1)

Question 3

Which parameter order is legal?

A. def f(a=1, b): B. def f(b, a=1): C. def f(*args, a): (without default — actually keyword-only, legal but advanced) D. Both B and C

Answer: D — but for this module level, B is the safe answer; C is keyword-only-after-args (legal). (L3)

Question 4

What's wrong with this?

```
def add_item(item, target=[]):  
    target.append(item)  
    return target
```

A. Syntax error B. Mutable default reused across calls C. Missing return D. Nothing

Answer: B (L3)

Question 5

What does *args collect arguments into?

A. A list B. A tuple C. A dict D. A set

Answer: B (L4)

Question 6

What does `**kwargs` collect arguments into?

A. A list B. A tuple C. A dict D. A set

Answer: C (L4)

Question 7

What happens here?

```
x = 5
def f():
    x = 10
f()
print(x)
```

A. 5 B. 10 C. None D. Error

Answer: A — inside `f` is a new local `x`. (L5)

Question 8

How do you modify a top-level variable from inside a function?

A. Use global VAR then reassign B. Use local VAR C. Reassign directly D. You cannot

Answer: A (L5)

Question 9

Which is a correct lambda for "square"?

A. `lambda x: return x x` B. `lambda x: x x` C. `lambda: x x` D. `def lambda x: x x`

Answer: B — no return, no def. (L6)

Question 10

How do you read a function's docstring?

A. `func.docs` B. `func.__doc__` C. `doc(func)` D. `func.help`

Answer: B — and also `help(func)`. (L6)

Module 4 — Assignment: Banking Operations

Estimated time: 1.5 hours

Combines: Module 4 + Modules 2-3

Brief

Build a set of pure functions that simulate banking operations. No classes (those come in Module 9) — just well-named functions you call in sequence.

Requirements

Implement these functions:

- `deposit(balance, amount)` → new balance.
- Reject negative amount; return original balance unchanged and print a warning.
- `withdraw(balance, amount)` → new balance.
- Reject negative amounts, and reject amounts greater than balance. Print warning, return unchanged.
- `interest(balance, rate=0.05, years=1)` → new balance ($\text{balance} \cdot (1 + \text{rate})^{\text{years}}$).
- `summary(balance, *transactions)` → prints account summary including each transaction and the final balance.
- Each function must have a one-line docstring.

Then write a small "main" section that:

- Starts with balance = 1000.
- Deposits 500, withdraws 200, applies interest at 8% for 3 years.
- Calls `summary()` with the running balance and a list of transaction descriptions.

Constraints

- Use only Module 1-4 concepts.
- Use `f"..."` strings for output.
- Round monetary output to 2 decimals.
- No classes, no lists/dicts in the function bodies (transactions can be `*args`).

Expected Behavior

```
Account Summary
-----
+500.00  deposit
-200.00  withdraw
+ interest at 8% for 3 years
-----
Final balance: 1632.66
```

(Math: $((1000 + 500 - 200) \cdot 1.083) = 1300 \cdot 1.259712 = 1637.62$ — your numbers will be exact when you compute them. Use whatever your code actually returns; the rubric checks the process, not a magic constant.)

Grading Rubric

Criterion	Weight
All 4 functions correct	40%
Docstrings present	10%
Validation (neg/over)	20%
Main flow + summary	20%
Style + naming	10%

Submission

Save as assignment_module4.py.

Module 4 — Mini Project: Modular Calculator

Overview

Refactor the Module-1 calculator into a clean function-based design. The same operations — but now each is its own function, with docstrings, defaults, and a single `run()` entry point.

Concepts Used

- `def`, return values (M4 L1-L2)
- Default args (M4 L3)
- Docstrings (M4 L6)
- `while`, `if/elif/else`, `match/case` (M3)

Features

- Functions: `add`, `subtract`, `multiply`, `divide`, `power`, `modulo`, `floor_div`.
- A `run()` function that:
- Prints a menu of operations.
- Asks for two numbers and the operation.
- Calls the right function and prints the result.
- Loops until the user picks "quit".
- Use `match/case` for dispatch.
- Each function has a one-line docstring.

Starter Code

```
# Mini Project: Modular Calculator

def add(a, b):
    """Return a + b."""
    return a + b

# TODO: implement subtract, multiply, divide, power, modulo, floor_div.
# divide should return None when b == 0 and the caller should print an error.

def run():
    """Interactive calculator loop."""
    while True:
        print("Operations: add | sub | mul | div | pow | mod | floor | quit")
        op = input("> ").strip().lower()
        if op == "quit":
            print("Bye!")
            break
        # TODO: ask for a and b, dispatch with match/case, print result.

if __name__ == "__main__":
    run()
```


Expected Interaction

```
Operations: add | sub | mul | div | pow | mod | floor | quit
> add
First number: 7
Second number: 3
Result: 10
Operations: add | sub | mul | div | pow | mod | floor | quit
> div
First number: 10
Second number: 0
Cannot divide by zero.
Operations: ... > quit
Bye!
```

Extension Challenges

- Add a history list (use a list; preview of Module 5) and a history command.
- Add support for negative-number formatting in output.
- Wrap `int(input(...))` so the user can retry on bad input — keep the loop alive.

Grading Rubric

Criterion	Weight
All operations as functions	30%
Docstrings present	10%
match/case dispatch	20%
Handles divide-by-zero	15%
Loop + quit works	15%
Style	10%